

Laporan Praktikum Minggu 6 - Navigasi Antar Layar

NAMA : HANIFAH HASANAH

NIM : 123140082

GITHUB :

https://github.com/HANIFAHHASANAH-123140082/123140082_Tugas6PAM_NewsReader.git

Link Video:

https://drive.google.com/file/d/1TbhqGdAr-6T04o003aD6B4BxqEHeJ7I1/view?usp=drive_link

A. Deskripsi Tugas Minggu 6:

Buat aplikasi "News Reader" dengan fitur:

1. Fetch berita dari public API (NewsAPI atau JSONPlaceholder)
2. Tampilkan list artikel dengan title, description, dan image
3. Detail screen saat artikel di-klik
4. Pull to refresh functionality
5. Proper loading, success, dan error states
6. Repository pattern untuk API calls

B. Tujuan

Membuat aplikasi mobile News Reader menggunakan React Native dan Expo yang dapat menampilkan daftar berita dari API publik, menampilkan detail berita, serta memiliki fitur pull to refresh dan penanganan berbagai kondisi seperti loading, sukses, dan error.

C. Langkah dan Penjelasan

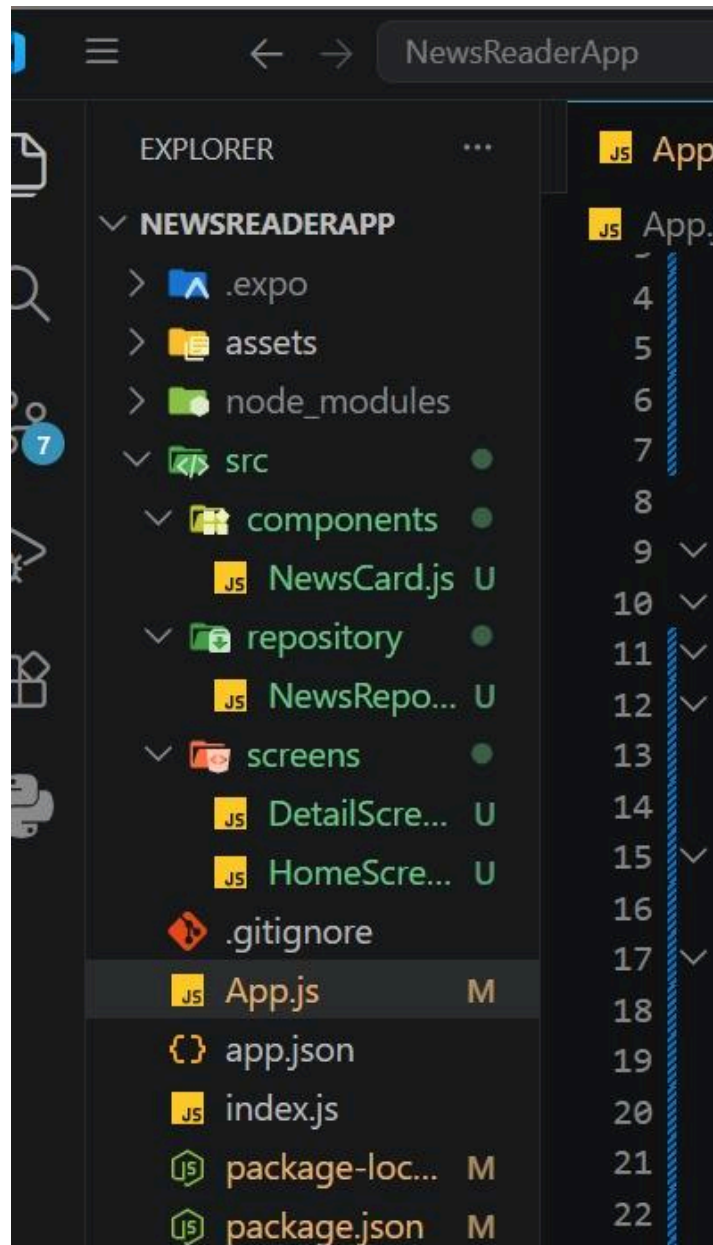
1. Membuat Project Baru

Project dibuat menggunakan perintah `npx create-expo-app NewsReaderApp --template blank` di terminal. Template blank dipilih agar project dimulai dari awal tanpa kode bawaan yang tidak diperlukan. Setelah project dibuat, library React Navigation juga diinstall menggunakan perintah `npm`

install@react-navigation/native@react-navigation/native-stack untuk mendukung perpindahan antar halaman.

2. Membuat Struktur Folder

Struktur folder dibuat di dalam folder src yang terdiri dari tiga subfolder yaitu screens untuk menyimpan halaman, repository untuk mengelola pemanggilan API, dan components untuk komponen yang bisa dipakai ulang. Struktur ini mengikuti pola Repository Pattern agar kode lebih rapi dan terorganisir.



3. Membuat NewsRepository.js

JavaScript

```
const BASE_URL = 'https://jsonplaceholder.typicode.com';

const NewsRepository = {
  getArticles: async () => {
    try {
      const response = await fetch(`${BASE_URL}/posts`);
      if (!response.ok) throw new Error('Failed to fetch');
      const data = await response.json();
      return data.slice(0, 20).map((item) => ({
        id: item.id,
        title: item.title,
        description: item.body,
        image: `https://picsum.photos/seed/${item.id}/400/200`,
        author: `Author ${item.userId}`,
        publishedAt: new Date().toLocaleDateString('id-ID'),
      }));
    } catch (error) {
      throw error;
    }
  },
};
```

Penjelasan:

Kode di atas menggunakan `fetch()` untuk mengambil data dari JSONPlaceholder API. Data yang diambil dibatasi 20 artikel menggunakan `slice(0, 20)`. Setiap artikel dilengkapi dengan gambar dari Picsum Photos berdasarkan ID artikel. Jika terjadi kesalahan saat mengambil data, error akan diteruskan ke halaman yang memanggilnya agar bisa ditampilkan pesan error kepada pengguna.

4. Membuat [NewsCard.js](#)

Komponen `NewsCard` dibuat sebagai tampilan kartu untuk setiap artikel berita di halaman utama. Komponen ini

menerima data artikel dan fungsi onPress sebagai props, sehingga bisa dipakai ulang untuk semua artikel.

JavaScript

```
const NewsCard = ({ article, onPress }) => {  
  return (  
    <TouchableOpacity style={styles.card} onPress={onPress}>  
      <Image source={{ uri: article.image }} style={styles.image}  
        resizeMode="cover" />  
      <View style={styles.content}>  
        <Text style={styles.title} numberOfLines={2}>{article.title}</Text>  
        <Text style={styles.description}  
          numberOfLines={3}>{article.description}</Text>  
        <View style={styles.footer}>  
          <Text style={styles.author}>{article.author}</Text>  
          <Text style={styles.date}>{article.publishedAt}</Text>  
        </View>  
      </View>  
    </TouchableOpacity>  
  );  
};
```

TouchableOpacity digunakan agar kartu bisa diklik dan memberikan efek visual saat ditekan. `numberOfLines={2}` pada judul membatasi teks maksimal 2 baris agar tampilan kartu tetap rapi. Gambar ditampilkan menggunakan komponen Image dengan sumber URL dari API.

5. Membuat [HomeScreen.js](#)

HomeScreen adalah halaman utama yang menampilkan daftar berita. Halaman ini mengelola tiga kondisi yaitu loading saat data sedang diambil, sukses saat data berhasil ditampilkan, dan error saat terjadi kegagalan koneksi.

JavaScript

```
const [articles, setArticles] = useState([]);  
const [loading, setLoading] = useState(true);  
const [refreshing, setRefreshing] = useState(false);
```

```
const [error, setError] = useState(null);

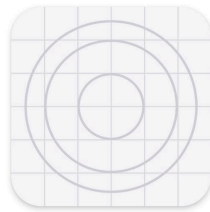
const fetchNews = async () => {
  try {
    setError(null);
    const data = await NewsRepository.getArticles();
    setArticles(data);
  } catch (err) {
    setError('Gagal memuat berita. Coba lagi.');
```

useState digunakan untuk menyimpan data artikel, status loading, status refresh, dan pesan error. Fungsi fetchNews memanggil repository untuk mengambil data. Jika berhasil, artikel disimpan ke state. Jika gagal, pesan error disimpan dan ditampilkan ke pengguna. Kata kunci finally memastikan status loading selalu dimatikan baik saat sukses maupun error.

10:03 | 0,5KB/d



Bundling 22,2%...



NewsReaderApp

Bundling

22,22%

6. Pull to Refresh

Fitur pull to refresh ditambahkan menggunakan komponen RefreshControl dari React Native. Pengguna cukup menarik layar ke bawah dari bagian atas daftar untuk memperbarui data berita.

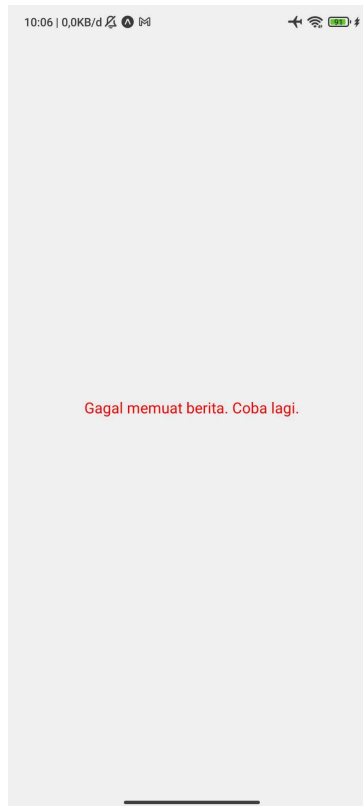
JavaScript

```
const onRefresh = useCallback(() => {
  setRefreshing(true);
  fetchNews();
}, []);

// Di dalam FlatList:
refreshControl={
  <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
}
```

useCallback digunakan agar fungsi onRefresh tidak dibuat ulang setiap kali komponen render. RefreshControl

dihubungkan ke FlatList sehingga saat pengguna menarik ke bawah, fungsi onRefresh akan dipanggil dan data diperbarui.



7. Membuat DetailScreen.js

DetailScreen menampilkan informasi lengkap dari artikel yang dipilih pengguna. Data artikel diterima melalui `route.params` yang dikirim dari HomeScreen saat kartu diklik.

JavaScript

```
const DetailScreen = ({ route }) => {
  const { article } = route.params;
  return (
    <SafeAreaView style={styles.container}>
      <ScrollView>
        <Image source={{ uri: article.image }} style={styles.image}
resizeMode="cover" />
        <View style={styles.content}>
          <Text style={styles.title}>{article.title}</Text>
          ...
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}
```

```
    </SafeAreaView>
  );
};
```

Data artikel yang diklik dikirim dari HomeScreen menggunakan `navigation.navigate('Detail', { article: item })` dan diterima di DetailScreen melalui `route.params`. `ScrollView` digunakan agar konten yang panjang bisa di-scroll oleh pengguna.



8. Mengatur Navigasi di App.js

`App.js` diatur sebagai file utama yang mengatur navigasi antar halaman menggunakan `React Navigation`. Terdapat dua halaman yaitu `Home` dan `Detail` yang dihubungkan menggunakan `Stack Navigator`.

JavaScript

```
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} options={{
headerShown: false }} />
        <Stack.Screen
```



```

        name="Detail"
        component={DetailScreen}
        options={{
          title: 'Detail Berita',
          headerStyle: { backgroundColor: '#007AFF' },
          headerTintColor: '#fff',
        }}
      />
    </Stack.Navigator>
  </NavigationContainer>
);
}

```

NavigationContainer membungkus seluruh aplikasi agar navigasi bisa bekerja. Stack.Navigator mengatur perpindahan halaman seperti tumpukan, dimana halaman baru ditumpuk di atas halaman sebelumnya. Header di halaman Home disembunyikan (headerShown: false) sedangkan halaman Detail menampilkan header berwarna biru dengan tombol back.



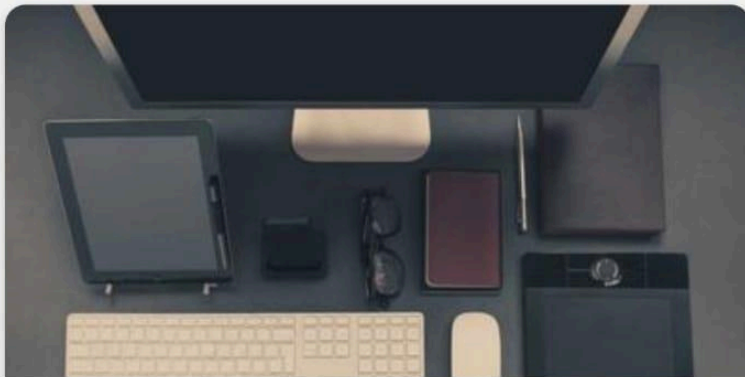
sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit

suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam...

[Author 1](#)

17/4/2026



qui est esse

est rerum tempore vitae

sequi sint nihil reprehenderit dolor beatae ea dolores neque...

[Author 1](#)

17/4/2026



D. Hasil

Aplikasi News Reader berhasil dibuat dan berjalan dengan baik. Aplikasi dapat menampilkan daftar 20 artikel berita yang diambil dari JSONPlaceholder API. Setiap artikel ditampilkan dalam bentuk kartu dengan gambar, judul, deskripsi singkat, nama penulis, dan tanggal. Saat artikel diklik, pengguna akan diarahkan ke halaman detail yang menampilkan informasi lengkap. Fitur pull to refresh berjalan normal, dan aplikasi dapat menangani kondisi error saat tidak ada koneksi internet.

E. Kesimpulan

Dari praktikum ini, dapat disimpulkan bahwa React Native dan Expo memudahkan pembuatan aplikasi mobile yang memiliki fitur lengkap. Penggunaan Repository Pattern memisahkan logika pengambilan data dari tampilan sehingga kode lebih terstruktur dan mudah dikembangkan. React Navigation memungkinkan perpindahan antar halaman dengan mudah, sementara state management menggunakan useState dan useEffect membantu mengelola berbagai kondisi tampilan aplikasi secara efisien.